

Windfarm Cable Routing

Problem 1

Sets

- $t \in T = \{1, 2, 3\}$: cable types
- $i, j \in \{1, 2, \dots, n, n + 1\}$: points where the n turbines are located and the substation being point $n + 1$

Parameters

- u_t : cost per meter of cable type t
- k_t : capacity of cable type t
- $DISTANCE_{i,j}$ distance between points i and j
- C : substation capacity
- $n + 1$: number of "points" (n turbines, 1 substation)

Decision variables

- $y_{i,j} \in \{0, 1\}$: is turbine i connected to (going into) turbine j via any cable (=1) or not (=0)
- $x_{i,j}^t \in \{0, 1\}$: is a cable of type t used to connect turbine i to j
- $f_{i,j} \geq 0$: Counting variable. Number of turbines supported by the connection between turbines i and j (counts turbine i but not turbine j).

Model

Objective:

- Minimize the total cabling costs :

$$\sum_{i=1}^{n+1} \sum_{j=1}^{n+1} \sum_{t \in T} u_t \cdot DISTANCE_{ij} \cdot x_{ij}^t$$

Constraints:

- Connect cable type variables, x , with turbine connection variables, y , such that if y -reflects the existence of a connection from i to j then exactly one cable type has been selected for that connection:

$$\sum_{t \in T} x_{ij}^t = y_{ij} \quad \forall i = 1, 2, \dots, n+1 \quad \forall j = 1, 2, \dots, n+1$$

- Balance constraint: Update the number of supported turbines (more details below):

$$\sum_{j=1}^{n+1} f_{ij} = \sum_{j=1}^n f_{ji} + 1 \quad \forall i = 1, \dots, n$$

- Cable type selected must be able to support the number of supported turbines for all connections:

$$\sum_{t \in T} k_t x_{ij}^t \geq f_{ij} \quad \forall i = 1, 2, \dots, n+1 \quad \forall j = 1, 2, \dots, n+1$$

- Each turbine must have one outgoing connection

$$\sum_{j=1}^{n+1} y_{ij} = 1 \quad \forall i = 1, 2, \dots, n$$

- Meanwhile, no connections out of the substation as this is "the end of the flow":

$$\sum_{j=1}^n y_{n+1,j} = 0$$

- Limit the maximum number of connections into the substation:

$$\sum_{i=1}^n y_{i,n+1} \leq C$$

Note: The constraint that keeps track of the supported turbines, the f -variables, is a kind of balance constraint: On the LHS we have the collective sum of supported turbines for all connections going into a turbine i , then we add 1 because that is turbine i itself. On the RHS we have the collective sum of supported

turbines on connections going out of i . There can only be one connection out of i , so the load will be on a single connection from i to some turbine/substation j , but by summing them all it is not necessary to know exactly which one for the constraint to work. Note that on the LHS the sum is only over turbines $j = 1, \dots, n$ as there cannot be a connection from the substation into turbine i , meanwhile, the connection out of i could be to the end-station so $j = n + 1$ is included in the sum on the RHS.

The full model in Julia/JuMP is given below

```

#####
# Windfarm Cable Routing "Mathematical Programming Modelling" (42112)
using JuMP
#using Gurobi
using HiGHS
#####

# number of turbines
n=34
# number of cable types
T=3

# scenario 1
u=[135, 250, 480]
k=[2, 6, 9]
C=4

#=
# scenario 2
u=[170, 290, 610]
k=[3, 7, 13]
C=3=#

#=
# scenario 3
u=[135, 290, 610]
k=[4, 6, 12]
C=3
=#

include("wind_farm_data.jl")

#WFCR=Model(Gurobi.Optimizer)
WFCR=Model(HiGHS.Optimizer)

# binary variable (cable type on edge)

```

```

@variable(WFCR, x[i=1:n+1,j=1:n+1,t=1:T], Bin)
# binary variable (edge between turbines selected or not)
@variable(WFCR, y[i=1:n+1,j=1:n+1], Bin)
# flow variable to count turbines "using" edge
@variable(WFCR, f[i=1:n+1,j=1:n+1]>=0)

# minimize cable costs
@objective(WFCR, Min, sum(u[t]*Distance[i,j]*x[i,j,t] for i=1:n+1 for j=1:n+1 for t=1:T))

# connection between x and y, if edge is selected then so is a cable type
@constraint(WFCR, [i=1:n+1,j=1:n+1], sum(x[i,j,t] for t=1:T) == y[i,j])
# turbine flow count
@constraint(WFCR, [i=1:n], sum(f[i,j]-f[j,i] for j=1:n+1) == 1)
# cable type capacity (cable type must be able to support the flow)
@constraint(WFCR, [i=1:n+1,j=1:n+1], sum(k[t]*x[i,j,t] for t=1:T) >= f[i,j])
# every turbine has one outgoing edge
@constraint(WFCR, [i=1:n], sum(y[i,j] for j=1:n+1)==1)
# no flow out of substation
@constraint(WFCR, sum(y[n+1,j] for j=1:n)==0)
# substation connection limit
@constraint(WFCR, sum(y[i, n+1] for i=1:n) <= C)

optimize!(WFCR)

if termination_status(WFCR) == MOI.OPTIMAL
    #Rounding the objective value to avoid scientific notation:
    println("Cable routing costs: $(round{Int,objective_value(WFCR)})")
    println("solution time: ", solve_time(WFCR) )
    println("Relative gap: ", relative_gap(WFCR) )
else
    println("No solution")
end

```

Note: To comment out a multi-line section of text, like the scenario data above, you can surround it by `#=` and `=#`.

Problem 2

New Parameters

- $ub = \lceil \frac{n}{C} \rceil$ Highest number of turbines a single root branch should carry in a balanced tree.
- $lb = \lfloor \frac{n}{C} \rfloor$ Lowest number of turbines a single root branch should carry in a balanced tree.

New Constraints

- Connections to the substation can support at most ub turbines

$$f_{i,n+1} \leq ub \quad \forall i = 1, 2, \dots, n$$

- connections to the substation must support at least lb turbines if there is a connection.

$$f_{i,n+1} \geq lb \cdot \sum_{t \in T} x_{i,j}^t \quad \forall i = 1, 2, \dots, n$$

Note: There is an important difference in the balance constraints for the lower and upper number of turbines. For the upper limit we can add the constraint for all possible connections regardless if whether the connection exists because in those cases $f_{i,n+1} = 0$ so the constraint is trivially satisfied. The same is not the case for the lower bound. Here we have to ensure that we only add the constraint $f_{i,n+1} \geq lb$ for connections that exist, otherwise we are saying that ALL turbines must connect to the substation with at least lb supported turbines and that is impossible.

The full model in Julia/JuMP is given below

```
*****
# Windfarm Cable Routing 2 "Mathematical Programming Modelling" (42112)
using JuMP
using Gurobi
using HiGHS
*****

# number of turbines
n=34
# number of cable types
T=3

# scenario 1
u=[135, 250, 480]
k=[2, 6, 9]
C=4

#=
# scenario 2
u=[170, 290, 610]
k=[3, 7, 13]
C=3=#

#=
# scenario 3
```

```

u=[135, 290, 610]
k=[4, 6, 12]
C=3
=#

include("wind_farm_data.jl")

#WFCR=Model(Gurobi.Optimizer)
WFCR=Model(HiGHS.Optimizer)

# binary variable (cable type on edge)
@variable(WFCR, x[i=1:n+1,j=1:n+1,t=1:T], Bin)
# binary variable (edge between turbines selected or not)
@variable(WFCR, y[i=1:n+1,j=1:n+1], Bin)
# flow variable to count turbines "using" edge
@variable(WFCR, f[i=1:n+1,j=1:n+1]>=0)

# minimize cable costs
@objective(WFCR, Min, sum(u[t]*Distance[i,j]*x[i,j,t] for i=1:n+1 for j=1:n+1 for t=1:T))

# connection between x and y, if edge is selected then so is a cable type
@constraint(WFCR, [i=1:n+1,j=1:n+1], sum(x[i,j,t] for t=1:T) == y[i,j])
# turbine flow count
@constraint(WFCR, [i=1:n], sum(f[i,j]-f[j,i] for j=1:n+1) == 1)
# cable type capacity (cable type must be able to support the flow)
@constraint(WFCR, [i=1:n+1,j=1:n+1], sum(k[t]*x[i,j,t] for t=1:T) >= f[i,j])
# every turbine has one outgoing edge
@constraint(WFCR, [i=1:n], sum(y[i,j] for j=1:n+1)==1)
# no flow out of substation
@constraint(WFCR, sum(y[n+1,j] for j=1:n)==0)
# substation connection limit
@constraint(WFCR, sum(y[i, n+1] for i=1:n) <= C)

# problem 2, balanced root branches:
ub = ceil(n/C) #upper number of turbines for root branches
lb = floor(n/C) #lower number of turbines for root branches

@constraint(WFCR, [i=1:n], f[i,n+1]<= ub)
@constraint(WFCR, [i=1:n], f[i,n+1]>= sum(lb* x[i,n+1,t] for t=1:T) )

optimize!(WFCR)

if termination_status(WFCR) == MOI.OPTIMAL
    #Rounding the objective value to avoid scientific notation:
    println("Cable routing costs: $(round{Int,objective_value(WFCR)})")
    println("solution time: ", solve_time(WFCR) )

```

```

println("Relative gap: ", relative_gap(WFCR) )
else
println("No solution")
end

```

Problem 3

In the solution to Scenario 3 turbine 21 is connected to turbine 13 and turbine 24 is connected to turbine 10. These connections cross each other.

New constraints

$$y_{10,24} + y_{24,10} + y_{13,21} + y_{21,13} \leq 1$$

and in general if connections (i,j) and (k,l) cross each other, the following constraint can be used

$$y_{i,j} + y_{j,i} + y_{k,l} + y_{l,k} \leq 1$$

You are welcome to check for crossing cables visually by e.g. drawing your solution. If you would like to see how it can be done in code you may take inspiration from the function "LineCrossChecker" which has been provided in the file `Line_Cross_Checker.jl` below the model.

The full model in Julia/JuMP is given below

```

#####
# Windfarm Cable Routing 3 "Mathematical Programming Modelling" (42112)
using JuMP
#using Gurobi
using HiGHS
#####

# number of turbines
n=34
# number of cable types
T=3

#=
# scenario 1
u=[135, 250, 480]
k=[2, 6, 9]
C=4=#

# =

```

```

# scenario 2
u=[170, 290, 610]
k=[3, 7, 13]
C=3=#

# scenario 3
u=[135, 290, 610]
k=[4, 6, 12]
C=3

include("wind_farm_data.jl")

#WFCR=Model(Gurobi.Optimizer)
WFCR=Model(HiGHS.Optimizer)

# binary variable (cable type on edge)
@variable(WFCR, x[i=1:n+1,j=1:n+1,t=1:T], Bin)
# binary variable (edge between turbines selected or not)
@variable(WFCR, y[i=1:n+1,j=1:n+1], Bin)
# flow variable to count turbines "using" edge
@variable(WFCR, f[i=1:n+1,j=1:n+1]>=0)

# minimize cable costs
@objective(WFCR, Min, sum(u[t]*Distance[i,j]*x[i,j,t] for i=1:n+1 for j=1:n+1 for t=1:T))

# connection between x and y, if edge is selected then so is a cable type
@constraint(WFCR, [i=1:n+1,j=1:n+1], sum(x[i,j,t] for t=1:T) == y[i,j])
# turbine flow count
@constraint(WFCR, [i=1:n], sum(f[i,j]-f[j,i] for j=1:n+1) == 1)
# cable type capacity (cable type must be able to support the flow)
@constraint(WFCR, [i=1:n+1,j=1:n+1], sum(k[t]*x[i,j,t] for t=1:T) >= f[i,j])
# every turbine has one outgoing edge
@constraint(WFCR, [i=1:n], sum(y[i,j] for j=1:n+1)==1)
# no flow out of substation
@constraint(WFCR, sum(y[n+1,j] for j=1:n)==0)
# substation connection limit
@constraint(WFCR, sum(y[i, n+1] for i=1:n) <= C)

# problem 2, balanced root branches:
ub = ceil(n/C) #upper number of turbines for root branches
lb = floor(n/C) #lower number of turbines for root branches

@constraint(WFCR, [i=1:n], f[i,n+1]<= ub)
@constraint(WFCR, [i=1:n], f[i,n+1]>= sum(lb* x[i,n+1,t] for t=1:T) )

```



```

# crossing example
@constraint(WFCR, y[10,24]+y[24,10]+y[13,21]+ y[21,13]<=1)

optimize!(WFCR)

if termination_status(WFCR) == MOI.OPTIMAL
    #Rounding the objective value to avoid scientific notation:
    println("Cable routing costs: $(round{Int,objective_value(WFCR)})")
    println("solution time: ", solve_time(WFCR) )
    println("Relative gap: ", relative_gap(WFCR) )
else
    println("No solution")
end

# Using code to check for crossings:
include("Line_Cross_Checker.jl")
println("\nChecking for crossings...")
for i=1:n
    for j=i:n+1
        for k=1:n
            for l=k:n+1
                if length(unique([i,j,k,l]))==4
if (value(y[i,j])>=0.01 || value(y[j,i])>=0.01
) && (value(y[k,l])>=0.01 || value(y[l,k])>=0.01)
crossflag,_= LineCrossChecker((xc[i],yc[i]),(xc[j],yc[j]),(xc[k],yc[k]),(xc[l],yc[l]))
if crossflag==true
    println("There is a cross between connection $i<->$j and $k<->$l")
end
end
end
end
end
end
end
end
println("DONE checking for crossings:")

```

Example code for how you can check for crossings:

```

*****
# Code by Kristine Børsting, 2026-01-09
# Made for the course 42112
*****

#The below function takes as input:

```

```

# four 2-dimensional points i,j,k,l (provide as coordinate tuples)
# and checks whether the lines from i->j and l->k crosses
# returning either true (they cross) or false (they do not cross)
# As well as the point the cross/would cross if the lines were extended
function LineCrossChecker(i::Tuple{Float64,Float64},j::Tuple{Float64,Float64},
                          k::Tuple{Float64,Float64},l::Tuple{Float64,Float64})
    # Creates the parameterised line from i->j and the one from l->k
    # Then sets up 2 equations with 2 unknowns to check if there exists point on both lines.
    # To solve the 2 equations I have rewritten them in matrix form:

    A=[1[1]-k[1] j[1]-i[1];
        1[2]-k[2] j[2]-i[2]] #LHS coefficients of equation system
    b=[j[1]-k[1];
        j[2]-k[2]] #RHS

    #Check if there exist values of the parameterization-variables such that points on both
    x = try A \ b
    catch e #The try/catch necessary in case the lines are parallel (above operation is ill)
        nothing
    end
    println("parameterisation values =",x)

    if isnothing(x) #Then the lines are parallel and never cross
        return false, nothing
    end

    #Calculate the point they cross based on the first line:
    CrossPoint=(k[1]+(1[1]-k[1])*x[1], k[2]+(1[2]-k[2])*x[1]) # x[1] is the value for the 1st line

    #They actually cross on the line segments
    if 1>=x[1]>=0 && 1>=x[2]>=0
        return true, CrossPoint
    else #they cross, but only if you extend the lines infinitely
        return false, CrossPoint
    end

end

end

#=
println("\n-----")
println("Test case 1: They DO cross:")
println("-----")
i=(0,0)
j=(1,1)
k=(1,0)
l=(0,1)

```

```

flag,point=LineCrossChecker(i,j,k,l)
println("Do they cross?\t\tflag. At: \tpoint")

println("\n-----")
println("Test case 2: They DO cross (in one of the points):")
println("-----")
i=(0,0)
j=(2,0)
k=(1,0)
l=(1,1)
flag,point=LineCrossChecker(i,j,k,l)
println("Do they cross?\t\tflag. At: \tpoint")

println("\n-----")
println("Test case 3: They do NO cross (but they do further out on the infite lines):")
println("-----")
i=(0,0)
j=(1,1)
k=(4,0)
l=(3,1)
flag,point=LineCrossChecker(i,j,k,l)
println("Do they cross?\t\tflag. At: \tpoint")

println("\n-----")
println("Test case 4: They do NO cross (ever, they are parallel):")
println("-----")
i=(0,0)
j=(0,1)
k=(1,0)
l=(1,1)
flag,point=LineCrossChecker(i,j,k,l)
println("Do they cross?\t\tflag. At: \tpoint")

=#

```